

Variance Soft Shadow Mapping: A WebGPU Implementation

FATEMEH SHIRVANI

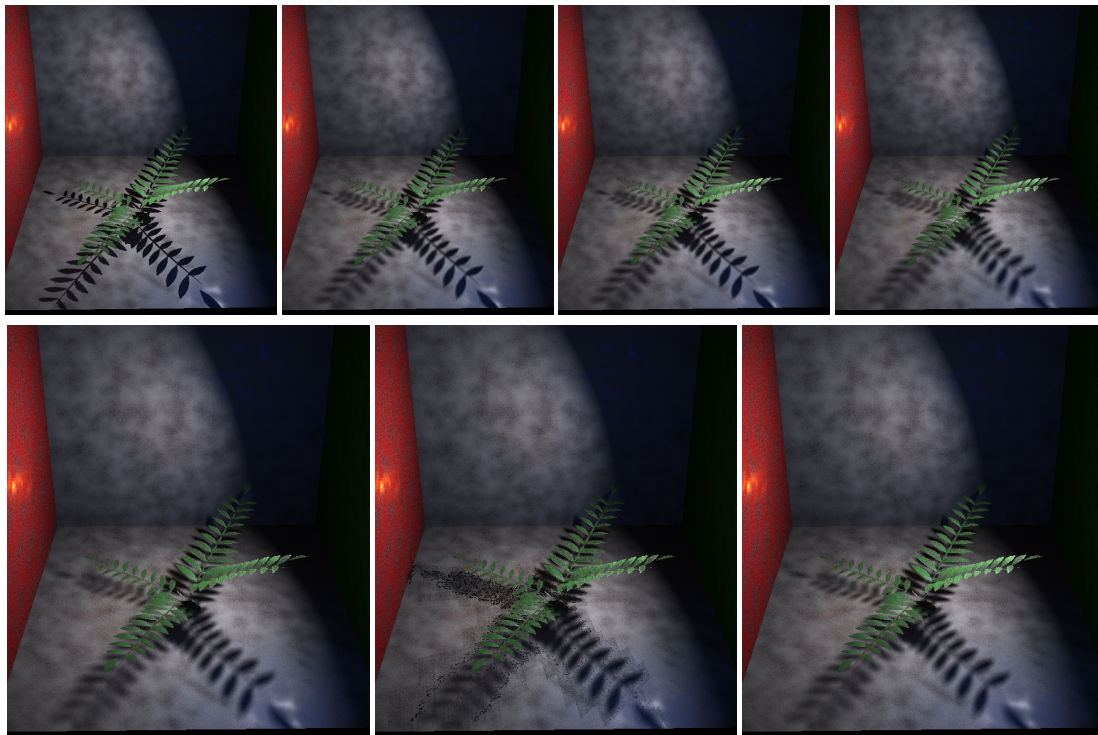


Fig. 1. Comparison of the implemented shadowing methods under identical camera, lighting, geometry, and material settings in the WebGPU application. Top row, left to right: Hard Shadow Map, PCF, VSM, and SAVSM. Bottom row, left to right: PCSS, VSSM Paper, and VSSM Hybrid.

This report presents a WebGPU implementation of Variance Soft Shadow Mapping (VSSM), based on the original algorithm of Yang et al. and implemented on top of the WebGPU assignment code provided for the Image Synthesis course at École Polytechnique. As in the original paper, the method combines ideas from percentage-closer soft shadows (PCSS) and variance shadow maps (VSM): a VSM-based blocker-depth estimate replaces the brute-force PCSS blocker search, while summed-area-table (SAT) moment filtering, min-max hierarchical shadow map (HSM) early-exit tests, and kernel subdivision are used to handle the *non-planarity* problem. During implementation, the main practical difficulty was the appearance of light leaking, bright spots, and instability on thin occluders and near contact-shadow regions. To address these cases, a second mode, denoted *VSSM Hybrid*, was added with adaptive quad-tree subdivision, exact PCF fallback in unstable kernels, and a contact-shadow correction path. The resulting web application runs entirely inside a single HTML file using WebGPU and WGSL, and retains the assignment ray tracing mode as a debugging reference.

1 INTRODUCTION

Soft shadows are an essential visual cue for scene realism. A physically accurate soft shadow arises when the light source is an area

emitter: each surface point may be partially occluded, creating a smooth transition from the fully-lit region through the penumbra into the hard umbra. Real-time rendering of soft shadows remains challenging because the standard algorithm — percentage-closer soft shadows (PCSS) [3] — requires brute-force point sampling of the shadow map in both a blocker-search pass and a soft-shadow-filter pass, with cost growing quadratically with filter radius. For large area lights or small blocker-to-receiver distances this becomes prohibitively expensive.

Variance shadow maps (VSM) [2] address the filtering cost by storing depth moments (z , z^2) in the shadow map, enabling arbitrary kernel averages in $O(1)$ time via pre-filtering or a summed-area table (SAT). The Chebyshev inequality then bounds the fraction of unoccluded texels. However, VSM applies a *single-bounded* reconstruction function that is most reliable when the local shadow kernel admits a simple planar interpretation. When this assumption breaks down — for instance, in a scene with blockers at varying depths — VSM can incorrectly illuminate points that should be shadowed, an artifact known as *non-planarity* light leaking.

Variance Soft Shadow Mapping (VSSM) [1] resolves this tension. Its key insight is a novel formula for estimating the average blocker depth using only the precomputed VSM moments (Equation 7),

Author's address: Fatemeh Shirvani, fatemeh.shirvani@ip-paris.fr.

eliminating the expensive brute-force blocker-search pass of PCSS. Whenever a kernel does not satisfy the mean-depth condition required by the VSM blocker estimate, it is treated as a non-planarity case and subdivided; variance is then used mainly as an auxiliary criterion inside the subdivision process. The result is a technique that runs at over 100 fps on complex scenes while producing shadow quality close to the ray-traced reference [1].

This report presents a WebGPU/WGSL implementation of VSSM embedded in a single HTML file and built on top of the WebGPU assignment code provided for the Image Synthesis course at École Polytechnique. The resulting application supports side-by-side comparison of Hard Shadow Maps, PCF, PCSS, VSM, SAVSM, VSSM Paper, and VSSM Hybrid, together with debug visualisations and a retained ray tracing mode used as a reference. In practice, the main implementation difficulty was the appearance of light leaking and instability on thin occluders and near contact-shadow configurations; this motivated the addition of a hybrid mode with adaptive subdivision and exact fallback sampling. Section 2 surveys related work; Section 3 details the full pipeline; Section 4 describes modifications and extensions beyond the paper; and Section 5 presents performance and quality observations.

2 RELATED WORK

Shadow maps. Williams [4] introduced shadow mapping: render depth from the light; during shading, compare the receiver depth against the stored depth. The technique is universal and GPU-friendly but produces aliased hard shadows.

Percentage-closer filtering. Reeves et al. [5] proposed PCF: instead of a single depth comparison, average the binary test over a neighbourhood of shadow-map texels. This produces anti-aliased shadow edges but does not model penumbras. Fernando [3] extended PCF to produce physically-motivated penumbras (PCSS) by computing the average blocker depth with a brute-force loop and scaling the filter radius proportionally. PCSS cost is $O(r_{\text{search}}^2 + r_{\text{filter}}^2)$, prohibitive for large lights.

Variance shadow maps. Donnelly and Lauritzen [2] observed that storing (z, z^2) in the shadow map and applying Chebyshev’s inequality yields a *pre-filterable* shadow test. Any axis-aligned rectangular filter can be evaluated in $O(1)$ using a SAT, as described by Crow [6]. The technique is fast but prone to light leaking when the planarity assumption is violated.

SAT-based variance shadow maps. Lauritzen [7] combined VSM with SAT for arbitrary-radius filtering, evaluating the blocker pass via brute-force on depth maps and the filter pass in $O(1)$ via VSM+SAT. However, SAVSM still inherits the light-leaking and multi-depth limitations of variance-based shadow reconstruction. The SAVSM mode in the implementation reproduces this baseline.

Variance Soft Shadow Mapping. Yang et al. [1] replaced the brute-force blocker search with a VSM-based estimator, and introduced kernel subdivision to suppress non-planarity artefacts, achieving over an order-of-magnitude speedup over PCSS for large penumbras.

3 TECHNICAL DETAILS

3.1 Pipeline Overview

The implementation follows the four-stage pipeline derived from Algorithm 1 of [1], translated into a sequence of WebGPU render and compute passes:

- (1) **Shadow map pass:** render the scene from the light’s perspective, writing a depth texture and a two-component moments texture $(z_{\text{lin}}, z_{\text{lin}}^2)$ stored in normalised form (see Section 3.2).
- (2) **SAT pass:** build a summed-area table over the moments texture via a compute-shader parallel prefix scan.
- (3) **HSM pass:** build a min-max hierarchical shadow map over the linearised depth texture via iterative 2×2 reduction compute shaders.
- (4) **VSSM shading pass:** for each visible fragment, execute the two logical phases of VSSM shading: first estimate the average blocker depth over the search kernel using Equation 7, then evaluate soft-shadow visibility over the penumbra filter kernel using the same precomputed data structures.

3.2 Moments Texture and Linearisation

Hardware shadow maps store NDC-space depth $d_{\text{ndc}} \in [0, 1]$. VSM requires linear-space moments so that the expected value $E[z]$ corresponds to a meaningful average distance. The moments texture stores normalised linear depth:

$$\tilde{z} = \frac{z_{\text{lin}} - z_{\text{near}}}{z_{\text{far}} - z_{\text{near}}}, \quad (1)$$

so both $\tilde{z}$ and $\tilde{z}^2$ lie in $[0, 1]$, fitting naturally in an `rgba32float` texture and keeping the per-texel moment values well conditioned before SAT accumulation. At query time, $E[z_{\text{lin}}]$ and $E[z_{\text{lin}}^2]$ are recovered by `denormalizeMomentsFromUnit`:

$$\begin{aligned} E[z] &= z_n + E[\tilde{z}] \Delta z, \\ E[z^2] &= z_n^2 + 2 z_n \Delta z E[\tilde{z}] + \Delta z^2 E[\tilde{z}^2], \end{aligned} \quad (2)$$

where $z_n = z_{\text{near}}$ and $\Delta z = z_{\text{far}} - z_{\text{near}}$.

3.3 Summed-Area Table Construction

The SAT is built by two sequential compute passes — one horizontal, one vertical — each applying a parallel Kogge-Stone prefix scan with $\log_2 N$ steps at stride 2^k , $k = 0, 1, \dots$. Each step dispatches a $(8, 8)$ workgroup grid that reads the source texture and adds the value at offset 2^k along the scan axis:

$$S[i] = S[i] + S[i - 2^k] \quad \text{if } i \geq 2^k. \quad (3)$$

A ping-pong buffer pair avoids read-write hazards. Conceptually, the final SAT enables the average moment over any axis-aligned rectangle $[u_1, u_2] \times [v_1, v_2]$ to be computed from four corner values:

$$\bar{m} = \frac{S[u_2, v_2] - S[u_1, v_2] - S[u_2, v_1] + S[u_1, v_1]}{(u_2 - u_1 + 1)(v_2 - v_1 + 1)}. \quad (4)$$

3.4 Min-Max Hierarchical Shadow Map

The HSM stores the minimum and maximum linearised depth in each mip level. Level 0 is produced by a base compute pass that linearises the hardware depth texture. Each subsequent level is

produced by an 8×8 workgroup reduction pass that downsamples 2×2 blocks:

$$\begin{aligned} \min_{l+1}(i, j) &= \min_{dx, dy \in \{0,1\}} \min_l (2i+dx, 2j+dy), \\ \max_{l+1}(i, j) &= \max_{dx, dy \in \{0,1\}} \max_l (2i+dx, 2j+dy). \end{aligned} \quad (5)$$

Given a filter kernel of radius r centred at $\mathbf{u}$, the implementation selects an appropriate mip level with $\lceil \log_2(2r) \rceil$ and samples a small neighbourhood of cells at that level to conservatively bound $[\min, \max]$ over the whole kernel. If the receiver depth $d_{\text{rec}} \leq \min$, the point is *fully lit*; if $d_{\text{rec}} > \max$, it is in the *umbra*.

3.5 VSSM Shading Pass

Inside shading, VSSM still follows the same two logical phases as PCSS, but it changes how both phases are evaluated. The first phase estimates the average blocker depth over the search kernel w_l ; the second phase uses that blocker depth to derive a penumbra radius and evaluate soft-shadow visibility over the filter kernel w_p . The main contribution of the paper is that the costly blocker-search phase is no longer performed by brute-force depth sampling, but instead by a closed-form estimate derived from VSM theory.

Given average moments $(E[z], E[z^2])$ over the search kernel w_l and the receiver depth t , Chebyshev's inequality gives:

$$P(z \geq t) \leq p_{\max}(t) = \frac{\sigma^2}{\sigma^2 + (t - \mu)^2}, \quad (6)$$

where $\mu = E[z]$ and $\sigma^2 = E[z^2] - \mu^2$. Treating $p_{\max}$ as the fraction of lit texels P_{lit} and defining $P_{\text{occ}} = 1 - P_{\text{lit}}$, the two-plane assumption gives:

$$\bar{z}_{\text{occ}} = \frac{\mu - P_{\text{lit}} \cdot t}{P_{\text{occ}}}. \quad (7)$$

This is implemented in `estimateBlockerDepthVSSM`. In the paper's formulation, the estimate is used when the search kernel satisfies the VSM mean-depth prerequisite $\mu < t$, under the corresponding two-plane approximation.

Once $\bar{z}_{\text{occ}}$ is estimated, the filter phase proceeds in the usual PCSS spirit: a penumbra radius is derived from the blocker-receiver distance, and visibility is reconstructed over the resulting soft-shadow kernel. In VSSM, however, this second phase is also accelerated by moment filtering, HSM early exits, and subdivision when the kernel violates the conditions under which the VSM approximation is reliable.

3.6 Non-Planarity Detection and Kernel Subdivision

A filter or blocker-search kernel becomes problematic when the VSM blocker estimate no longer satisfies its mean-depth prerequisite. In the notation used here, the estimate is reliable when the average depth $\mu = E[z]$ remains below the receiver depth t . When $\mu \geq t$, the kernel is treated as a *non-planarity* case and subdivided. The variance threshold Θ is still exposed in the implementation, but it is used mainly as an auxiliary classifier inside the subdivision process rather than as the primary definition of non-planarity.

Blocker-search subdivision. If the whole-kernel mean depth satisfies $\mu \geq t$, the search kernel w_l is uniformly subdivided into $m \times m$ sub-kernels. For each sub-kernel:

- If the sub-kernel still satisfies the VSM mean-depth prerequisite $\mu < t$: apply Equation 7 using SAT moments.
- If the sub-kernel is still unstable after subdivision: fall back to a 3×3 PCF sample grid to obtain the blocker depth directly from the raw depth texture.

The per-sub-kernel weighted average yields the final $\bar{z}_{\text{occ}}$.

Soft-shadow subdivision. A symmetric subdivision is applied to the penumbra kernel w_p in `filterShadowVSSMPaperResult`. At this stage, mean depth and variance are both used to decide whether a sub-kernel can remain on the VSM path or should fall back to a small exact PCF evaluation. The final visibility is the area-weighted average over all sub-kernels. Algorithm 1 summarises the procedure.

Algorithm 1 VSSM soft-shadow filter (mode 7)

Require: shadowCoord $\mathbf{s}$, receiver depth t , filter radius r , threshold Θ

- 1: $K \leftarrow$ SAT moments over $[\mathbf{s} \pm r]$
 - 2: **if** $E_K[z] < t$ **and** $\sigma_K^2 \leq \Theta$ **then return** $p_{\max}(t; E_K[z], \sigma_K^2)$ ▷ whole-kernel VSM fast path
 - 3: Subdivide $[\mathbf{s} \pm r]$ into $m \times m$ sub-kernels $\{w_i\}$
 - 4: **for each** w_i **do**
 - 5: $(\mu_i, \sigma_i^2) \leftarrow$ SAT moments of w_i
 - 6: **if** $\mu_i \geq t$ **then** $V_i \leftarrow 1$ ▷ lit-classified sub-kernel in the current implementation
 - 7: **else if** $\sigma_i^2 > \Theta$ **then** $V_i \leftarrow \text{PCF}(w_i)$ ▷ non-planarity
 - 8: **else** $V_i \leftarrow p_{\max}(t; \mu_i, \sigma_i^2)$ ▷ normal VSM
 - return** $\sum_i |w_i| V_i / \sum_i |w_i|$
-

3.7 Contact Shadow Correction

Following Section 6.4 of [1], when the estimated blocker depth satisfies $|t - \bar{z}_{\text{occ}}| \leq \epsilon_c$ (contact threshold), the receiver and blocker are nearly co-planar and both the planarity assumption and the SAT precision are unreliable. In this case a 3×3 jittered bilinear PCF is applied directly, avoiding the precision artefacts visible in contact regions. Figure 2 shows the same local crop with this correction disabled and enabled.

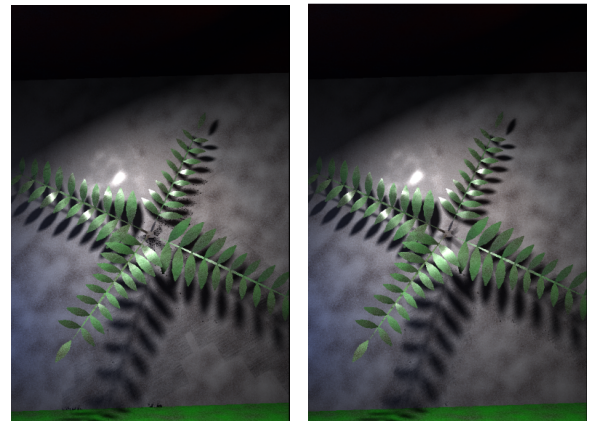


Fig. 2. Same local crop with contact correction disabled (left) and enabled (right). The correction stabilises the receiver-blocker contact region.

4 MODIFICATIONS AND IMPROVEMENTS

4.1 Adaptive Quad-Tree Subdivision (VSSM Hybrid)

In addition to the uniform $m \times m$ subdivision of the paper, an *adaptive* variant (mode 6) is implemented, inspired by Algorithm 2 of [1]. The hybrid path keeps a uniform $m \times m$ subdivision for blocker-depth estimation, but replaces the final visibility filter with an explicit stack-based quad-tree traversal (depth ≤ 4). Each filter node is classified as:

- *VSM accept*: $\sigma^2 \leq \Theta$, node depth-range entirely before the receiver with a positive safety margin, range span compact, and Chebyshev value near 0 or 1.
- *PCF leaf*: maximum recursion depth or 1×1 pixel — exact PCF with sub-pixel coverage weighting.
- *Subdivide*: otherwise, push four children onto the stack.

This hybrid organisation keeps the blocker search close to the paper-oriented path while concentrating final-filter sample budget in high-variance penumbra regions and avoiding unnecessary subdivision in flat lit or shadow regions.

4.2 Linear Depth Moments with Normalised Storage

A subtle engineering choice is that the moments texture stores *normalised* linear depths (Equation 1) rather than raw linear depths. This keeps both channels of the `rgba32float` texture in $[0, 1]$, reducing the risk of precision loss in the SAT accumulation for deep scenes and making the SAT seed pass trivial (a simple clamp and copy). The denormalisation in Equation 2 is applied at query time at negligible cost.

4.3 Multi-Technique Split-Screen Comparison Interface

The web application provides a split-screen rendering with independently configurable left and right views (Figure 3). Both halves share the same scene and camera matrix but have separate `ShadowUniform` buffers, allowing any two techniques from the following list to be compared side-by-side: None, Hard Shadow Map, PCF, PCSS, VSM, SAVSM, VSSM Hybrid, VSSM Paper. Shadow parameters such as bias and filter radius are controlled per side, while shared VSSM and lighting controls expose area-light size, subdivision level, and variance threshold for comparative evaluation.

4.4 Debug Modes

Four debug modes are available per view:

- (1) **Shadow factor**: grey-scale visibility $V \in [0, 1]$.
- (2) **VSSM branches**: RGB encodes $(V_{\text{VSM}}, V_{\text{lit}}, V_{\text{PCF}})$ fraction per pixel, making the subdivision classification directly visible.
- (3) **Moment / Chebyshev**: $(\mu_{\text{norm}}, \sigma_{\text{norm}}^2, p_{\text{max}})$ per pixel, useful for diagnosing non-planarity.
- (4) **SAT delta**: absolute error between SAT-based and brute-force moment estimates, useful for detecting precision loss in large kernels.

5 OBSERVATIONS AND FUTURE WORK

5.1 Shadow Quality

The teaser (Figure 1) shows the expected ordering of methods. Hard Shadow Maps expose shadow-map aliasing; PCF removes aliasing

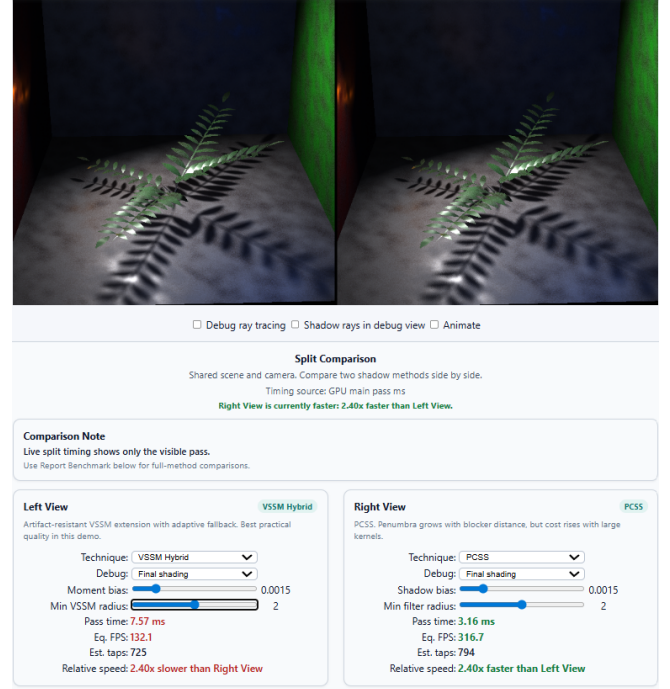


Fig. 3. Split-screen WebGPU application used throughout the project. The two panels render the same scene and camera configuration while exposing independent technique selection and per-view shadow parameters, enabling controlled visual and performance comparisons between PCSS, VSM-family methods, and the two VSSM variants.



Fig. 4. Comparison with the original VSSM paper. Left: example reproduced from [1]. Right: output of the WebGPU VSSM Hybrid mode. Both illustrate the intended soft-penumbra reconstruction on thin foliage.

but replaces contact hardening with an almost uniform blur; PCSS restores penumbra growth with blocker-receiver separation, although small kernels still show visible quantisation.

The variance-based methods separate more clearly on the plant scene. VSM and SAVSM remain plausible on broad, nearly planar receivers, but once a kernel spans several depth layers they exhibit the over-lit regions identified in the original paper. VSSM Paper (mode 7) mitigates this failure through subdivision and fallback, recovering a penumbra profile close to PCSS. VSSM Hybrid (mode 6) is the most stable practical variant in the current implementation, especially on thin foliage and near-contact regions.

5.2 Performance

Algorithmically, the paper’s performance argument still applies: PCSS pays a brute-force blocker-search cost that grows with the search radius, whereas VSSM replaces that stage with moment queries and hierarchy tests. In practice, however, the WebGPU implementation also adds SAT and HSM prepasses, so only full-method totals would support a meaningful quantitative comparison. In practice, stable end-to-end totals were not recorded consistently enough to support a strong numerical speedup claim in this implementation.

5.3 Effect of Variance Threshold

The variance threshold Θ is an important quality knob of the implementation. Internally, it influences the boundary between the VSM fast-path and the PCF fallback in Algorithm 1, but it should not be read as the primary non-planarity definition of the original paper. On the blocker-estimation side, the central failure case is the mean-depth relation $E[z] \geq t$; the variance threshold is better understood as an auxiliary control for subdivision and acceleration.

At $\Theta = 0$, nearly every ambiguous sub-kernel is routed to the fallback path, so mode 7 behaves like a conservative subdivided filter. At very large values, the method degenerates toward plain VSM and recovers its characteristic over-lit regions. The paper’s default value, $\Theta = 5 \times 10^{-4}$, remains a reasonable operating point in this implementation. It also interacts with the subdivision level m : finer subdivision produces smaller, more nearly planar sub-kernels, allowing slightly larger thresholds without obvious loss of quality. In practice, the *SAT delta* debug mode is useful for separating variance-driven leakage from SAT-precision error.

5.4 Limitations

Single-depth shadow map. The implementation renders a single depth map from the light centre. Multiple depth layers are partially handled by subdivision, but kernels containing three or more distinct layers may still underestimate blocker depth because Equation 7 is derived under a two-plane assumption.

SAT precision at large radii. The SAT uses `rgba32float` storage, so large-radius queries can lose precision during the four-corner subtraction in Equation 4. The *SAT delta* debug mode exposes this directly. The paper recommends a 32-bit integer SAT, which could be emulated in WebGPU via `rg32uint` fixed-point storage.

Area light model. Penumbra width is computed under the thin-lens / area-disc approximation. The actual penumbra boundary for an extended rectangular light source differs from the circular approximation, especially at grazing angles. More accurate models based on solid-angle integration exist [3] but require per-fragment integration over the light shape.

5.5 Comparison with the Original Paper

Table 1 summarises which components of the original VSSM paper are reproduced in this implementation. The core paper pipeline is present in mode 7: shadow map, linear moments, SAT, min-max hierarchy, blocker-depth estimation from moments, and non-planarity handling through subdivision and fallback. The main deliberate divergence is organisational rather than algorithmic: the adaptive strategy associated with Algorithm 2 of [1] is exposed as the separate

Table 1. Coverage of the main VSSM [1] components. ✓ = implemented; × = not implemented.

Component	Status
Shadow map + linear moments	✓
Summed-area table (SAT)	✓
Min-max HSM early exit	✓
VSM blocker depth formula (Eq. 7)	✓
Uniform $m \times m$ kernel subdivision	✓
Non-planarity PCF fallback (filter)	✓
Non-planarity PCF fallback (blocker search)	✓
Contact shadow correction	✓
Adaptive quad-tree subdivision	✓ (mode 6)
Penumbra width formula (PCSS)	✓
32-bit integer SAT	×

VSSM Hybrid mode instead of as an internal option of mode 7, which keeps the paper-oriented and practical variants directly comparable in the split-screen interface.

Future extensions include cascaded shadow maps to tighten the depth range, integer SAT storage to improve large-radius precision, and bounded alternatives such as Exponential Shadow Maps [8].

6 CONCLUSION

This project presents a substantial WebGPU implementation of Variance Soft Shadow Mapping [1] embedded in a single HTML file. Mode 7 reproduces the core paper pipeline: moments, SAT, min-max hierarchy, blocker-depth estimation from moments, subdivision with fallback, and contact shadow correction. Mode 6 adds a more practical hybrid filter with adaptive subdivision and exact leaf evaluation for greater stability on thin foliage and near-contact regions.

The resulting application supports direct comparison with Hard Shadow Maps, PCF, PCSS, VSM, and SAVSM, together with debug modes and a retained ray tracing reference inherited from the Image Synthesis assignment codebase. Remaining limitations are consistent with both the paper and the present WebGPU design, notably the single-depth shadow-map model, floating-point SAT precision, and the simplified area-light approximation.

REFERENCES

- [1] Yang, B., Dong, Z., Feng, J., Seidel, H.-P., and Kautz, J. “Variance soft shadow mapping.” *Computer Graphics Forum* 29, 7 (2010), 2249–2258. Pacific Graphics 2010.
- [2] Donnelly, W. and Lauritzen, A. “Variance shadow maps.” In *Proceedings of the 2006 Symposium on Interactive 3D Graphics and Games*, ACM, 2006, pp. 161–165.
- [3] Fernando, R. “Percentage-closer soft shadows.” In *ACM SIGGRAPH 2005 Sketches*, ACM, 2005, p. 35.
- [4] Williams, L. “Casting curved shadows on curved surfaces.” In *Proceedings of SIGGRAPH 1978*, ACM, 1978, pp. 270–274.
- [5] Reeves, W. T., Salesin, D. H., and Cook, R. L. “Rendering antialiased shadows with depth maps.” In *Proceedings of SIGGRAPH 1987*, ACM, 1987, pp. 283–291.
- [6] Crow, F. C. “Summed-area tables for texture mapping.” In *Proceedings of SIGGRAPH 1984*, ACM, 1984, pp. 207–212.
- [7] Lauritzen, A. “Summed-area variance shadow maps.” In *GPU Gems 3*, Nguyen, H. (ed.), Addison-Wesley, 2007, ch. 8.
- [8] Annen, T., Mertens, T., Seidel, H.-P., Flerackers, E., and Kautz, J. “Exponential shadow maps.” In *Proceedings of Graphics Interface 2008*, 2008, pp. 155–161.